Programación para Ingeniería Telemática
Curso 2026-27

Práctica 2: Herencia, versiones.

1. Objetivos de aprendizaje. Estructura de la práctica. 1
2. Clonación de un repositorio remoto. 3
3. El código de partida. 4
4. Creación de ramas. Refactorización del código. 5
5. Herencia, composición y patrones. 9
5.1. El patrón Estrategia: 9
5.2. Cálculo del precio del alquiler. 10
5.3. El Método Factoría. 11
5.4. Serialización y deserialización de objetos. Factoría para la deserialización de vehículos 11
6. Ejercicios 14
7. Recursos. 15
8. Realización, entrega y evaluación 15

1. Objetivos de aprendizaje. Estructura de la práctica.

Objetivos de la práctica.

- Presentar nuevas funcionalidades de Git (`clone`, `push`, `pull`, `branch`, `switch`, `merge`).
- Repasar conceptos básicos de la programación orientada a objetos.
- Introducir los patrones de diseño.

Objetivos de aprendizaje.

Ser capaces de:

- Clonar un repositorio remoto.
- Subir versiones del repositorio local al repositorio remoto (`push`) y viceversa (`pull`, `fetch`).
- Crear ramas (`branch`) para el desarrollo o mantenimiento de diferentes partes del proyecto y cambiar de una rama a otra (`switch`, `checkout`).
- Integrar ramas secundarias en la rama principal (`merge`).
- Desarrollar de forma colaborativa e incremental un pequeño proyecto software.
- Identificar problemas que se pueden resolver aplicando los patrones de diseño Estrategia y Factoría.

Contenidos de la memoria.

Esta memoria se estructura en # secciones con los siguientes contenidos:

Sección 1: Objetivos y contenido de la práctica.

Sección 2: Clonación de un repositorio remoto.

Sección 3: Explicación del código de partida.

Sección 4: Creación de ramas. Refactorización del código. Esta sección es una **práctica dirigida** en la que vamos a crear una rama auxiliar y realizar modificaciones en la rama auxiliar y en la principal, y después fusionarlas.

Sección 5: Herencia, composición y patrones. Esta sección presenta los patrones de diseño Estrategia y Método Factoría y algunos casos de aplicación que los estudiantes tendrán que desarrollar en los ejercicios propuestos.

Sección 6: Ejercicios.

Sección 7: Recursos.

Sección 3: Realización, entrega y evaluación..

2. Clonación de un repositorio remoto.

Aunque el código inicial de la práctica está disponible en el aula virtual, **vamos a crear un nuevo proyecto con el código de partida clonando un repositorio remoto**. Para abra en VS Code el directorio del que desee que cuelgue el proyecto y ejecute desde consola el siguiente comando:

```
git clone https://github.com/japf0666/p2-inicial.git
```

Observará que aparece una estructura de directorios correspondiente a un proyecto Maven que contiene el código de partida que más adelante comentaremos.

Cuando clonamos un repositorio remoto, obtenemos un repositorio local que contiene todas las versiones almacenadas en el repositorio remoto. Además, Git crea en el repositorio local una variable llamada `origin` que apunta al repositorio remoto desde el que hemos clonado. Esta variable se usa en las operaciones para subir información desde el repositorio local al remoto (`push`) o para traer información desde el remoto al local (`pull`, `fetch`).

El repositorio remoto desde el que hemos clonado (`p2-inicial`) es un repositorio "archivado", lo que significa que no admite nuevas actualizaciones (operaciones `push`). Para poder guardar en un repositorio remoto las modificaciones que vayamos haciendo localmente en nuestro proyecto **necesitamos asociar nuestro repositorio local con un repositorio remoto diferente**. Para ello:

- Elijan o creen un repositorio remoto vacío en su cuenta de GitHub.
- Cambien el valor de la variable `origin` para que apunte a dicho repositorio. Para ello basta con ejecutar en la consola de VS Code el siguiente comando:

  `git remote set-url origin https://github.com/user/repositorio.git`

  donde `user` y `repositorio` se corresponden con su nombre de usuario y con el nombre del repositorio que han creado. En todo caso, la URL pueden obtenerla copiándola del repositorio remoto de GitHub.

- Envíen el contenido de su repositorio local al nuevo repositorio remoto.

  `git push origin main`

Observaciones:

- `origin` es el nombre que se usa convencionalmente para referirnos al repositorio remoto principal asociado a un repositorio local.
- Cuando se ejecuta `git clone`, Git crea automáticamente un remoto llamado `origin` que apunta al repositorio del que se ha realizado la clonación. Ese nombre se utiliza después en órdenes como `git fetch origin`, `git pull origin main` o `git push origin main` para indicar con qué repositorio remoto se quiere intercambiar información.
- Sin embargo, `origin` no es una palabra reservada de Git ni una característica específica de GitHub: es un alias configurable que puede cambiarse, eliminarse o coexistir con otros remotos, como `upstream`.

3. El código de partida.

La organización del código de partida se muestra en el diagrama de clases de la figura 1 que expresa las relaciones entre clases usando el lenguaje UML[1]. Cada clase se representa por un rectángulo dividido en tres sectores correspondientes al nombre de la clase (superior), atributos de la clase (intermedio) y métodos de la clase (inferior). Los métodos y atributos privados van precedidos por un signo – y los públicos por un signo +. Los métodos y atributos subrayados son estáticos. La herencia se expresa mediante una flecha desde la subclase a la superclase y la composición mediante un rombo en la parte del agregado. Se puede añadir la cardinalidad de la composición.

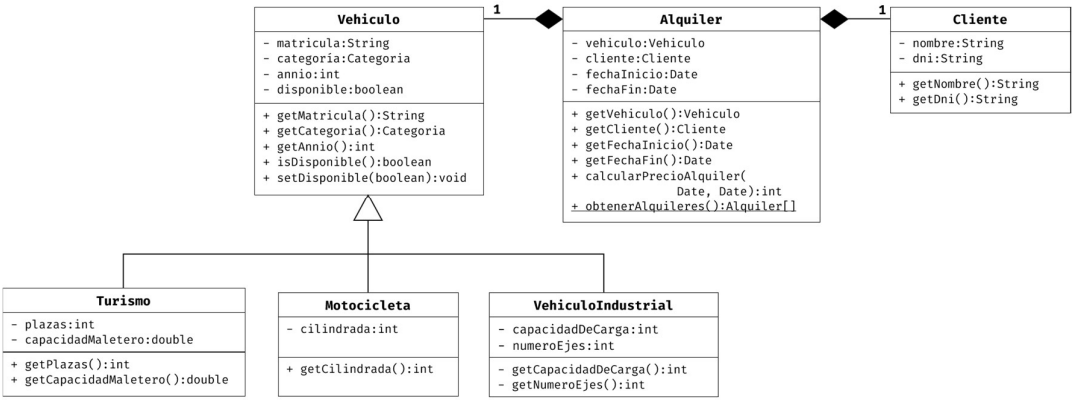


Figura 1: Sistema de alquiler de vehículos. Diagrama de clases.

Las clases e interfaces de la figura 1 se encuentran tanto en el Aula Virtual como en el repositorio remoto que hemos clonado.

Hay otras tres clases y un fichero que se encuentran sólo en el Aula Virtual y que se comentarán según se vayan presentando:

- La clase `KeyBoard` que servirá para recoger datos desde el teclado.
- El tipo enumerado `PoliticasDePrecios`.
- La clase `GestorAlquileres` que modela una pequeña aplicación de consola interactiva desde la que simula el proceso de alquiler de un vehículo.
- Un fichero `.gitignore`.

[1] UML, Unified Modelling Language. Es el lenguaje estándar para expresar el diseño del software. Iremos introduciendo elementos de UML a medida que se vayan necesitando.

4. Creación de ramas. Refactorización del código.

Las ramas (branches) se utilizan habitualmente para desarrollar nuevas funcionalidades, corregir errores o experimentar con cambios de forma aislada. Una vez finalizado y validado el trabajo, los cambios de una rama pueden integrarse en otra mediante una operación de fusión de ramas (`merge`). Cada rama representa una evolución del proyecto a partir de un punto determinado del historial, de modo que es posible realizar cambios en una rama sin afectar al estado de otras ramas.

Como ejemplo de creación y fusión de ramas, vamos a llevar a cabo una primera refactorización del código de partida en una rama separada. **Para crear una nueva rama usamos el comando `branch`:**

```
git branch refactor_fechas
```

Este comando crea la rama, pero no nos coloca en ella[2]. **Para cambiar de rama usamos el comando `switch`:**

```
git switch refactor_fechas
```

Si ahora introducen el comando **`git status`** podrán observar que nos hemos situado en la nueva rama:

```
C:\ directorio_proyecto >git status
On branch refactor_fechas

nothing added to commit
```

Vamos a realizar un "pequeño" cambio en esta rama. En lugar de utilizar la clase `Date` para modelar las fechas en la clase `Alquiler` vamos a utilizar `LocalDate`, ya que `LocalDate` nos va a permitir manejar la duración de los alquileres de forma más sencilla. Aunque el cambio parece menor, nos va a obligar a realizar los siguientes cambios en la clase `Alquiler`[3]:

- Importar las clases `LocalDate` y `ChronoUnit`:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
```

- Cambiar todas las variables y parámetros del tipo `Date` a `LocalDate`.
- Cambiar la implementación de `calcularPrecioAquiler` para que sea coherente con los métodos de `LocalDate`:

```
public int calcularPrecioAlquiler(LocalDate fechaInicio,
                                  LocalDate fechaFin) {
   // +1 to include the start day
   int dias = (int) ChronoUnit.DAYS.between(fechaInicio, fechaFin) + 1;
   return dias * PRECIO_BASE;
}
```

[2] Es posible crear la rama y cambiarse a ella en un solo paso mediante `git checkout nombre_de_la_rama`.
[3] Es muy habitual que lo que a priori consideramos un pequeño cambio propague sus efectos por todo el código.

- Cambiar la implementación de `main`, que quedaría así:

```
public static void main(String[] args) {
   Vehiculo vehiculo1 = new Turismo("ABC123",Categoria.NORMAL,2020,5,400.0);
   Cliente cliente1 = new Cliente("Juan Pérez", "123456789");
   LocalDate hoy = LocalDate.now();
   Alquiler alquiler1 =
          new Alquiler(vehiculo1, cliente1, hoy, hoy.plusDays(3));
   System.out.println(alquiler1.toString());
   System.out.println("-----------------------------------------------");

   Vehiculo vehiculo2 =
          new VehiculoIndustrial("ABC123", Categoria.NORMAL, 2020, 1000, 3);
   Alquiler alquiler2 =
          new Alquiler(vehiculo2, cliente1, hoy, hoy.plusDays(3));
   System.out.println(alquiler2.toString());
   System.out.println("-----------------------------------------------");
 }
```

Una vez realizados los cambios y comprobado que todo el código compila y que el método `main` se ejecuta sin errores vamos a:

- Añadir un fichero `.gitignore` a la raíz de proyecto (utilice el disponible en el Aula Virtual).
- Añadir los ficheros que queremos que se tengan en cuenta en `commit`

```
git add src
git add pom.xml
```

- **Realizar un `commit` en la rama `refactor_fechas`:**

```
git commit -m “cambio fechas a LocalDate en clase Alquiler”
```

- **Cambiar a la rama principal para trabajar en ella.**

```
git switch main
```

Si inspecciona ahora los contenidos del fichero `Alquiler` verá que están tal y como estaban. Las modificaciones se han realizado sobre la versión de la rama `refactor_fechas`, no sobre la rama `main`.

Vamos a realizar ahora otra modificación sobre el software de la rama principal, que consiste en añadir a `Vehiculo` dos métodos abstractos: `getTipo` y `getPrecioBase`.

El método abstracto `getTipo`, que nos va a facilitar más adelante la deserialización de objetos, se define de la siguiente manera:

```
/**
 * En serialización/deserialización: para obtener el tipo de
 * vehículo (clase) del objeto.
 * @return El tipo de vehículo (sub-clase) del objeto.
 */
 public abstract String getTipo();
```

La definición de este método en `Alquiler` nos va a obligar a proporcionar una implementación en las subclases, que consistirá simplemente en devolver el nombre de la subclase. Por ejemplo, en `Turismo`:

```
public String getTipo(){
    return "Turismo";
}
```

Al incluir este método, la librería `json.org` incluye un campo en la representación JSON del objeto con el nombre de la clase del objeto. Este campo nos va a dar un criterio para poder reconstruir el objeto serializado a partir de su representación JSON.

El método abstracto `getPrecioBase` nos va a permitir definir un precio base de alquiler para cada tipología de vehículo (turismo, motocicleta o vehículo industrial) en función de su categoría (normal, especial o superior)[4]: La inclusión de este método en `Vehiculo` nos va a facilitar el cálculo de los alquileres en la clase `Alquiler`.

```
/**
  * Calcula el precio base diario del vehículo según su categoría y tipo.
  * @return precio base diario del vehículo.
  */
public abstract int getPrecioBase();
```

La definición de este método en `Vehiculo` nos va a obligar a proporcionar una implementación en las subclases. Por ejemplo, en `Turismo`:

```
@Override
    public int getPrecioBase() {
        int precioBase = 0;
        switch (getCategoria()) {
            case NORMAL:
                precioBase = 50;
                break;
            case ESPECIAL:
                precioBase = 80;
                break;
            case SUPERIOR:
                precioBase = 100;
                break;
        }
        return precioBase;
    }
```

Nuevamente, ejecutamos el método `main` de la clase `Alquiler` y si todo va bien, añadimos los ficheros fuente y `pom.xml` al *stage* área y realizamos un `commit`

```
c:\directorio_proyecto>git add src
c:\ directorio_proyecto>git add pom.xml
c:\ directorio_proyecto>git commit -m "Añadidos métodos getTipo y
getPrecioBase en Vehiculo y sus subclases"
```

[4] Este diseño tiene algunos inconvenientes, pero para los propósitos de la práctica lo podemos dar por bueno de momento, y nos puede servir de ejemplo de los compromisos a los que hay que llegar durante el diseño.

En este punto tenemos dos ramas sobre las que hemos hecho modificaciones, la rama `refactor_fechas` y la rama `main`. Para fusionar dos ramas tenemos que estar situados en la rama que recibirá los cambios, en nuestro caso la rama principal, y ejecutar el comando **`merge`** indicando la rama que queremos fusionar. En nuestro caso ya estamos en `main`, por lo que podemos invocar `merge` de la siguiente manera:

```
git merge refactor_fechas
```

La fusión se realiza sobre un commit previo. Es posible que se nos solicite un `commit` para confirmar la fusión.

```
git commit -m "fusion main con fechas_refactor"
```

Si inspeccionamos el código vemos que la nueva versión recoge tanto los cambios que hemos realizado en la rama principal como en la auxiliar. En este caso ha sido muy sencillo porque no hay inconsistencias entre los cambios realizados en ambas versiones. Cuando Git detecta inconsistencias nos muestra ambas versiones y nos pregunta con cual nos quedamos (dejamos esto para otras prácticas).

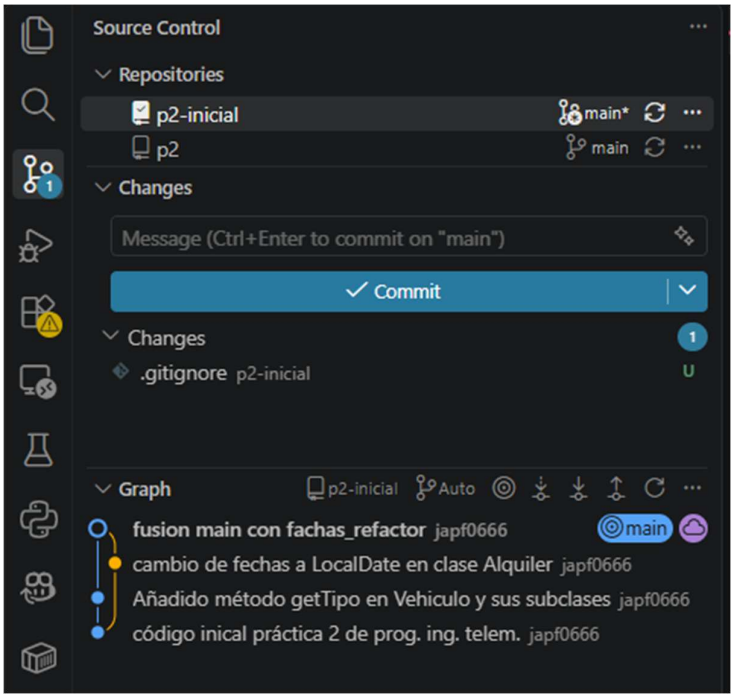


Figura 2: Grafo con historial de versiones y fusionado de ramas.

5. Herencia, composición y patrones.

Un patrón de diseño es una **solución general, reutilizable y contrastada para resolver un problema recurrente en el diseño de software**. En programación orientada a objetos, los patrones de diseño describen cómo organizar las clases y los objetos para conseguir sistemas más flexibles, mantenibles y extensibles. Un patrón de diseño **no proporciona código directamente reutilizable, sino un esquema de solución** que debe adaptarse a las características y necesidades de cada aplicación.

En esta práctica vamos a emplear dos patrones de diseño muy comunes[5] que nos van a proporcionan un marco en el que definir jerarquías de herencia y composición. Como se muestra en la figura 3, los patrones de diseño van un paso más allá que los principios de diseño. Los principios son aplicables en cualquier tipo de situación, mientras que los patrones son aplicables dentro de un contexto en el que proporcionan una solución probada a un cierto tipo de problema de programación. En este sentido, **un punto clave de los patrones es definir su aplicabilidad**.

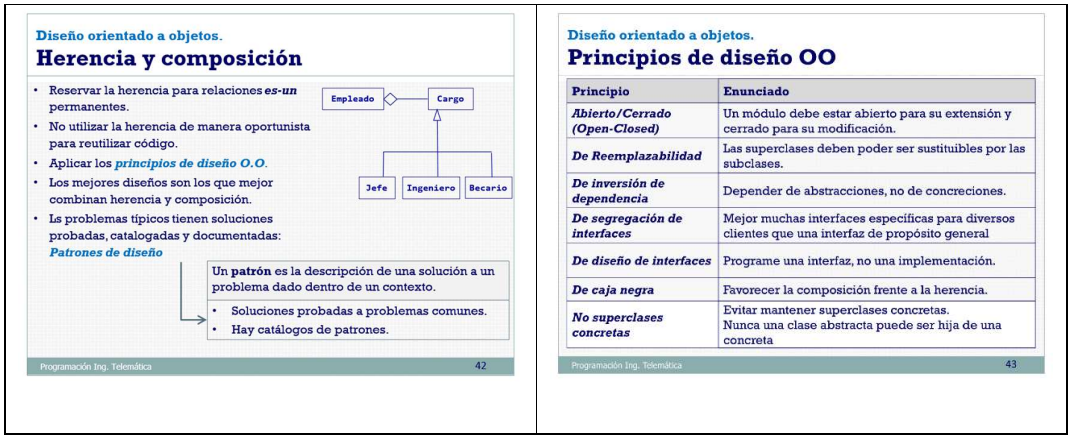


Figura 3: principios y patrones de diseño OO.

5.1. El patrón Estrategia:

El patrón Estrategia (figura 4) **permite a una clase cliente (el contexto) elegir entre varios algoritmos (o varias implementaciones de un mismo algoritmo) que comparten la misma interfaz** sin necesidad de utilizar sentencias condicionales. Mediante el patrón Estrategia podemos encapsular cada una de las implementaciones y hacerlas intercambiables. El patrón permite que el algoritmo varíe independientemente de los clientes que lo utilizan.

El patrón Estrategia se clasifica como un ***patrón de comportamiento*** porque permite cambiar dinámicamente el comportamiento de un objeto cambiando su estrategia. Puede haber cualquier número de estrategias, se pueden añadir más y cualquiera de ellas se puede modificar en cualquier momento sin afectar al contexto. Son ejemplos típicos de uso de este patrón guardar archivos en diferentes formatos, comprimir archivos con diferentes algoritmos, capturar datos de vídeo con diferentes esquemas

[5] Todos los patrones de diseño son usos comunes, si no, no serían patrones, pero unos son más comunes que otros.

de compresión y representar gráficamente los mismos datos en diferentes formatos: gráfico de líneas, gráfico de barras o gráfico circular.

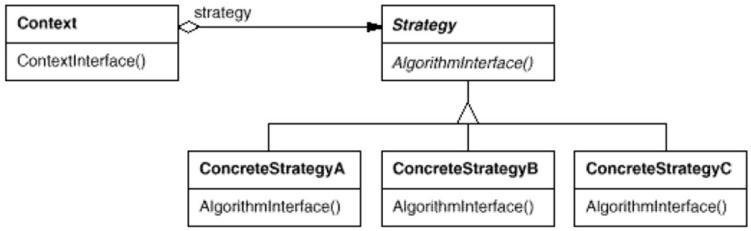


Figura 4: estructura del patrón Estrategia. GAMMA, Erich, et al. *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH, 1995.

5.2.Cálculo del precio del alquiler.

En esta práctica vamos a utilizar el patrón Estrategia para definir diferentes políticas de cálculo del precio del alquiler, como se muestra en la figura 5. Se van a definir las siguientes políticas de precios:

- Política normal: precio base x número de días de alquiler.
- Política temporada alta: se duplica el precio base diario.
- Política ofertón: se aplica un 30% de descuento al precio base y si el alquiler es de más de siete días se aplica además al total un descuento adicional del 10%.
- Política cliente frecuente: Se aplica al precio base un descuento del 20% los tres primeros días de alquiler y un descuento del 40% a partir del cuarto día.

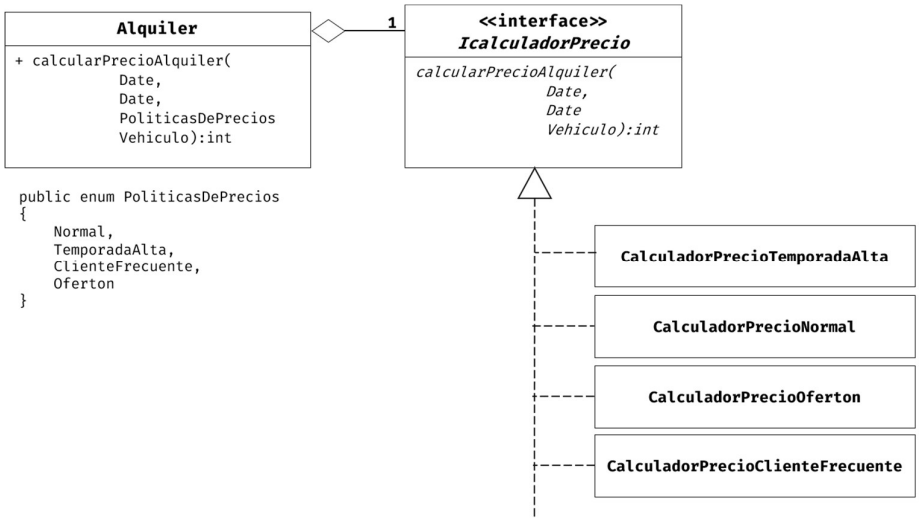


Figura 5: Políticas de precios aplicables al alquiler de un vehículo.

El contexto es la clase alquiler, que seleccionará o creará un calculador de precio en función de la política de precios aplicable en cada caso.

5.3.El Método Factoría.

En Java la forma más sencilla de crear un objeto es mediante el operador `new` y un constructor. Sin embargo, es frecuente que la naturaleza exacta del objeto que se debe crear varíe y con ella el constructor a utilizar y los valores de sus argumentos. En estas circunstancias, encapsular el proceso de creación en una clase o función "creadora" hace que el programa sea más flexible y general. En este apartado vamos a ver la versión más sencilla de las factorías, el **Método Factoría** (figura 6).

El Método Factoría devuelve un objeto de una de las posibles subclases de una clase base dependiendo de los datos que se le proporcionen. Normalmente, todos los objetos que devuelve pertenecen a una misma jerarquía de herencia (patrón estrategia implícito). El patrón Método Factoría encapsula tanto la lógica de selección de la subclase del objeto como la de selección del constructor más adecuado y se usa típicamente cuando una clase no puede anticipar la (sub)clase de objetos que debe crear.

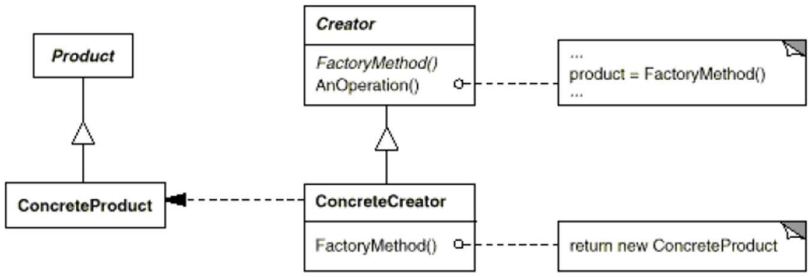


Figura 6: Patrón Método Factoría. GAMMA, Erich, et al. *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH, 1995.

En esta práctica vamos a usar este patrón para crear objetos a partir de su representación JSON y para crear políticas de precios a partir de su denominación.

5.4. Serialización y deserialización de objetos. Factoría para la deserialización de vehículos.

La serialización consiste en transformar un objeto en una representación que pueda almacenarse en un fichero o transmitirse a través de una red. **La deserialización** es el proceso inverso: reconstruir un objeto a partir de esa representación. Estos mecanismos permiten guardar y recuperar el estado de los objetos, así como intercambiar información entre aplicaciones.

El formato JSON (JavaScript Object Notation) es especialmente conveniente para serializar objetos porque es sencillo, legible para un humano, independiente del lenguaje de programación y ampliamente utilizado. Además, existen bibliotecas que facilitan la conversión entre objetos y JSON, aunque la reconstrucción de objetos complejos puede requerir información adicional sobre sus clases.

En el código de esta práctica, la serialización se hace en formato JSON usando la librería `org.json`. **Serializar es muy sencillo**, ya que basta con pasar el objeto a serializar por el constructor de `JSONObject`. En el código aportado esto se hace en los métodos `toString`.

```
@Override
public String toString() {
    return new JSONObject(this).toString();
}
```

Si ejecutan los métodos `main` de las clases `Cliente` y `Alquiler` podrán ver las representaciones JSON correspondientes a diferentes tipos de objetos. Por ejemplo, para `Cliente` se obtiene:

```
{"nombre":"Juan Pérez","dni":"12345678A"}
```

Y para la clase `Alquiler` (los saltos de línea, la indentación y la negrita se han añadido para facilitar la lectura y no están en el JSON original):

```
{
"cliente":{"nombre":"Juan Pérez","dni":"123456789"},
"fechaInicio":"2026-10-09",
"vehiculo":{"tipo":"Turismo",
            "precioBase":50,"categoria":"NORMAL",
            "matricula":"ABC123","plazas":5,"annio":2020,
            "capacidadMaletero":400,"disponible":true},
"fechaFin":"2026-10-12"
}
```

La representación JSON de `Alquiler` nos permite observar que:

- Se ha construido un JSON que respeta la estructura interna de `Alquiler`, serializándose todos los objetos que contiene.
- Se ha incluido un campo para todos los atributos para los que existe un método `get`*`Atributo`*.
- En el caso del vehículo se incluye el campo `tipo` con valor `Turismo` porque los vehículos tienen un método `getTipo`. No ocurre lo mismo con `Cliente` ni con `Alquiler` donde no se ha definido el método `getTipo`.

Para deserializar los objetos, es decir para construir un objeto a partir de su representación JSON, podemos definir un constructor que tome como argumento el objeto JSON que lo representa. Por ejemplo, en el caso de `Vehiculo`:

```
public Vehiculo(String jsonString) {
    JSONObject json = new JSONObject(jsonString);
    this.matricula = json.getString("matricula");
    this.categoria = Categoria.valueOf(json.getString("categoria"));
    this.annio = json.getInt("annio");
    this.disponible = json.getBoolean("disponible");
}
```

El objeto `JSONObject` ofrece métodos suficientes para ir extrayendo la información (pueden ver los detalles en la documentación de `json.org.JSONObject`). En los constructores de las subclases basta con invocar el constructor de la superclase y añadir con la información específica. Por ejemplo, en `Motocicleta`:

```
    public Motocicleta(String jsonString) {
        super(jsonString);
        JSONObject json = new JSONObject(jsonString);
        this.cilindrada = json.getInt("cilindrada");
    }
```

El problema es que cuando recibimos un objeto JSON no sabemos a priori a que subclase pertenece y por tanto no sabemos qué constructor invocar. Si tuviéramos que recuperar objetos serializados de un flujo de entrada (p.e.: de un fichero o de un enlace de comunicaciones) tendríamos que leer el JSON y deducir de su contenido la subclase a la que pertenece para llamar al constructor adecuado. Esto se soluciona incluyendo en el JSON información que nos permita determinar fácilmente su clase (en nuestro caso incluyendo un método `getTipo`) y creando una factoría de objetos (código de abajo) que determine por nosotros el constructor a invocar y lo llame retornando el objeto correspondiente.

Observe que en el método `main` de `FactoriaVehiculos` recreamos el objeto serializado directamente a través del método `getVehiculo`, sin preocuparnos por la lógica de deserialización, que en este caso es muy sencilla, pero que en otros casos podría ser más complicada. Observe también que podríamos añadir más subclases de vehículo sin que el código que usa la factoría se viera afectado.

```
public class FactoriaVehiculos {

    public static Vehiculo getVehiculo(String jsonString) {
        JSONObject json = new org.json.JSONObject(jsonString);
        String clase = json.getString("tipo");
        switch (clase) {
            case "Turismo":
                return new Turismo(jsonString);
            case "Motocicleta":
                return new Motocicleta(jsonString);
            case "VehiculoIndustrial":
                return new VehiculoIndustrial(jsonString);
            default:
                throw new IllegalArgumentException(
                            "Tipo de vehículo desconocido: " + clase);
        }
    }

    public static void main(String[] args) {

        Vehiculo v1 = new Turismo("1234ABC", Categoria.NORMAL, 2020, 5, 450.0);
        String serializado = v1.toString()
        Vehiculo v2 = FactoriaVehiculos.getVehiculo(serializado);
    }
}
```

6. Ejercicios.

1. El código proporcionado no incorpora test unitarios. **Añadan tests para comprobar sistemáticamente su funcionalidad.** Los métodos `main` son muy útiles para ciertas comprobaciones, pero no pueden sustituir a las pruebas sistemáticas.

2. **Modifique o extienda el código para incorporar el patrón estrategia en el cálculo de los precios de los alquileres, tal y como se explica en el apartado 5.2**.

   Resuelva este apartado en una **rama separada**, incluya **pruebas sistemáticas** e incorpore un pequeño **ejemplo de uso** en un método `main`. Una vez terminado este ejercicio, **fusione la rama** creada para resolverlo con la rama principal.

3. **Modifique o extienda el código para incorporar el patrón método factoría para la serialización y deserialización de clientes, vehículos y alquileres según las indicaciones del apartado 5.4**.

   Resuelva este apartado en una **rama separada**, incluya **pruebas sistemáticas** e incorpore un pequeño **ejemplo de uso** en un método `main`. Una vez terminado este ejercicio, **fusione la rama** creada para resolverlo con la rama principal.

4. **Modifique o extienda el código para incorporar el patrón método factoría para crear diferentes políticas de precios**.

   Resuelva este apartado en una **rama separada**, incluya **pruebas sistemáticas** e incorpore un pequeño **ejemplo de uso** en un método `main`. Una vez terminado este ejercicio, **fusione la rama** creada para resolverlo con la rama principal.

5. **Extienda el código de la clase `GestorAlquileres` para (1) guardar los objetos de la aplicación en formato JSON en un fichero de texto y para (2) recuperar los objetos guardados en un fichero e incorporarlos a la aplicación**.

   Este apartado puede resolverse en la rama principal o en una rama separada, pero modifique exclusivamente a la clase `GestorAlquileres`.

   Incluya pruebas sistemáticas y extienda la interfaz de usuario de `GestorAlquileres` para poder guardar y recuperar datos de ficheros de texto.

6. **Complete la clase `GestorAlquileres` (métodos "por realizar")**.

   Este apartado puede resolverse en la rama principal o en una rama separada, pero modifique exclusivamente a la clase `GestorAlquileres`..

Observaciones:

- **Cada vez que terminen un apartado realicen un commit del código obtenido y súbanlo a su repositorio remoto.**
- **Las pruebas sistemáticas y los métodos `main` pueden realizarse con la IA** a través de un prompt. Compruebe, no obstante, la calidad del código generado y si no la considera aceptable mejore el código a mano o vuelva a interactuar con la IA hasta obtener un resultado satisfactorio.

7. Recursos.

- Las clases `Cliente`, `Vehiculo`, `Turismo`, `Motocicleta`, `VehiculoIndustrial`, `Categoria` y `Alquiler`: en el Aula Virtual y en el repositorio remoto.
- Las clases `KeyBoard`, `PoliticasDePrecios` y `GestorAlquileres`: sólo en el Aula Virtual.

8. Realización, entrega y evaluación.

- La práctica se realizará por parejas.
- Los miembros de la pareja podrán repartirse el trabajo como consideren oportuno. En la entrevista de evaluación tendrán que justificar la división del trabajo.
- Cada miembro de la pareja responderá tanto de su parte del trabajo como de la colaboración con su compañero. La colaboración tiene que basarse tanto en la responsabilidad personal para realizar la parte del trabajo que a cada miembro de la pareja le corresponde como en la ayuda mutua.
- Fecha límite de entrega: 2/11/26, a las 23:55. **Se subirá al aula virtual un enlace al repositorio remoto en el que se han ido subiendo las soluciones de los diferentes apartados**.
- La evaluación se realizará mediante entrevista en la sesión de prácticas del 06/11/26.

Criterios de evaluación:

- Los apartados 1 y 6 valen un punto, el resto dos puntos.
- Conformidad con el diseño (uso de los patrones): 30%
- Corrección funcional: 30%. Incluye las pruebas sistemáticas y la calidad de los métodos `main`.
- Control de versiones: 20%. Uso de ramas, fusiones, progresión de las versiones, interacción con el repositorio remoto.
- Reparto del trabajo: 20%. Se valorarán la división equilibrada del trabajo y la colaboración. En caso de observarse un desequilibrio significativo se reflejará en las calificaciones de los miembros del equipo.
- Ética:
  - Si en la entrevista resulte evidente que la calidad del trabajo no se corresponde con la autoría la calificación será CERO.
  - El uso de la IA está explícitamente permitido para la generación de las pruebas sistemáticas y de los métodos `main` demostrativos.
  - Si se usa la IA en otros apartados, habrá que demostrar en la entrevista que se entiende el código generado, explicándolo, dando una visión crítica del mismo y respondiendo a las preguntas de los profesores. De no ser así, resultará evidente que la autoría corresponde a la IA y no a los estudiantes entrevistados.